

MAROS SCANNER Project DNA

Master Architecture Document — Vision, Stack, Features, Integrations & Design System

Master Document v2.0 • June 02, 2026

1. Vision & Purpose

Maros Scanner is a personal professional-grade stock trading platform built by Oscar Rodriguez. It combines real-time market scanning, AI-powered analysis, automated trading execution, and a full trade journal — all in one bilingual (English/Spanish) web application.

Target user: An active retail trader who wants institutional-quality tools without a \$3,500/month Bloomberg subscription. The app is designed to eventually become a SaaS product offered by subscription to other traders.

2. Core Principles

- Mobile First: Every feature works on iPhone 16 Pro (402x874 CSS px) and Pixel 10 XL. Desktop is enhanced, not primary.
- No Fake Data: Every price, score, and signal comes from a real API. No mock data in production.
- Safari Safe: All JavaScript is compatible with Safari/WebKit. Banned: `new Date(localeString)`, `findLastIndex`, `.at(-n)`, `structuredClone`, `Object.hasOwn`.
- No Browser Storage: Zero use of `localStorage`, `sessionStorage`, or `indexedDB`. All state lives on the server (SQLite) or in React Query cache.
- Bilingual: Every UI string is translated in English and Spanish via the `useLanguage()` hook. Lang toggle is always visible.
- Paper First: Auto-trader defaults to paper mode. Real money requires explicit user action.

3. Technology Stack

Layer	Technology	Notes
Frontend Framework	React 18 + TypeScript	SPA with hash routing (Wouter)
Build Tool	Vite 7	Hot reload dev, optimized prod bundles
UI Components	shadcn/ui + Tailwind CSS v3	Custom dark theme, teal accent
State / Data	TanStack React Query	Server sync, staleTime controls, auto-refetch
Routing	Wouter with useHashLocation	Hash-based — required for SPA on pplx.app
Backend Framework	Express.js (Node.js)	REST API server, port 5000
Language (Server)	TypeScript compiled to CJS	tsx for dev, esbuild for prod
Database	SQLite via Drizzle ORM	Synchronous <code>.get()/all()</code> calls. <code>data.db</code> in root
Hosting	pplx.app (Perplexity Computer)	Auto-sandbox, wakes on request, persists DB

CI/CD	Manual build + deploy	NODE_ENV=production npm run build, then publish
-------	-----------------------	---

4. API Integrations

API	Tier	Rate Limit	Key Endpoints Used
Alpaca Paper	Free	Unlimited paper	Orders, positions, account, price feeds
Alpaca Data	Free	200 req/min	Screener, movers, bars, quotes
TwelveData	Free	8 credits/min, 800/day	RSI, MACD, EMA, VWAP, quote, price
Finnhub	Free	60 req/min	Insider transactions, earnings calendar
Gemini AI	Free tier	Per-model limits	Frozita AI coach (gemini-2.5-flash)
Yahoo Finance	Free (unofficial)	No official limit	Historical data, fundamentals
SEC EDGAR	Free	10 req/sec	Insider filings fallback (RSS feed)

5. App Navigation Structure

Mobile Bottom Nav (always visible — 6 items)

Position	Label	Icon	Route	Description
1	Picks	Star	/	Daily AI-graded stock picks (A/B/C)
2	Swing	BarChart2	/swing	Multi-day swing trade setups
3	Live	Radio	/livepicks	Real-time scanner picks (updates live)
4	Insider	Eye	/insider	Insider buying transactions (Finnhub)
5	InPlay	Zap	/inplay	Stocks with active price alerts firing
6	More	MoreHorizontal	Drawer	Opens tools/portfolio drawer

More Drawer Contents

Morning Prep, Gems/Potential, Scanner, Watchlist, Journal, Frozita AI, Taxes

Desktop Sidebar Groups

TRADING: Picks, Swing, Live Picks, Insider Trading, Potential, In Play

TOOLS: Morning Prep, Scanner, Watchlist

PORTFOLIO: Journal, Tax Inventory

FROZITA: AI Coach (highlighted in amber)

6. Key Server Files

File	Purpose
server/routes.ts	All API endpoints (~3000 lines). Central nervous system.
server/twelvedata-service.ts	TwelveData: RSI/MACD/EMA/VWAP, regime, scoring, catalyst, RS.
server/auto-trader.ts	Auto-trade engine: regime block, earnings block, grade sizing, brackets.
server/auto-trader-poller.ts	Runs every 5 min. Reads live cache, calls evaluateAndTrade().
server/alpaca-price.ts	Live price fetch from Alpaca Data API. Pre-trade verification.
server/insider-engine.ts	Finnhub insider buys (Code P), sorted by dollar. SEC RSS fallback.
server/storage.ts	SQLite interface. All read/write via Drizzle ORM (synchronous).
server/hermes.ts	Hermes Brain: self-learning weight optimizer by win/loss dimension.
script/build.ts	Build script. Compiles client + server. Copies .env to dist/.
scripts/qa-check.mjs	Pre-deploy QA: 22 checks — Safari, storage, nav, regime guards.

7. Design System

Theme: Dark mode only. Background #0a0a0a, surface cards with border #1a1a1a. Primary accent: Teal hsl(162 84% 39%). Score badges: A=green, B=amber, C=red.

Typography: System sans-serif stack. Monospace (font-mono class) for all prices and scores.

Routing: Wouter with useHashLocation hook. Router uses hook={useHashLocation} — not Switch.

i18n: Every component calls const { t, lang } = useLanguage() at the top of the function. Never outside.

Modal z-index: Minimum z-[60] for any modal or overlay. Never lower.

Devices tested: iPhone 16 Pro (402x874 CSS px), Pixel 10 XL (user primary), Desktop Windows.